

Docker Real-Time Interview Preparation

What is Docker?

Docker is a containerization platform used to package application code, dependencies, libraries, and runtime into a single portable unit called a container.

It solves:

- “Works in my machine” issue
 - Environment mismatch
 - Dependency conflicts
 - Deployment inconsistency
-

Why Docker?

Before Docker:

Developer System → Java 11
Server → Java 8

Application fails in production.

With Docker:

Same container runs everywhere:

Laptop
Testing Server
Production
Cloud
Kubernetes

Benefits of Docker

- Lightweight
- Fast startup

- Portable
 - Easy scaling
 - Better resource utilization
 - Consistent environments
 - Works well with Kubernetes
-

How to Explain in Interview

“Docker is a containerization platform that packages application code along with dependencies and runtime libraries into portable containers. In my projects, we used Docker to standardize deployments across development, QA, and production environments, which eliminated environment mismatch issues and improved deployment consistency.”

Docker vs Virtual Machine

Docker	Virtual Machine
Shares host kernel	Separate OS
Lightweight	Heavy
Fast startup	Slow
Less resource usage	High resource usage
Container isolation	Full OS isolation

Docker Architecture (Very Important)

Core Components

Docker Client



Docker Daemon (dockerd)



Container Runtime



Components Explanation

Docker Client

Where user executes commands:

docker build
docker run
docker ps

Docker Daemon (dockerd)

Background service that:

- Builds images
 - Runs containers
 - Manages networking
 - Manages volumes
-

Docker Images

Read-only template used to create containers.

Example:

nginx image
ubuntu image
java image

Docker Container

Running instance of image.

Example:

docker run nginx

Docker Registry

Stores images.

Examples:

- DockerHub
 - AWS ECR
 - Azure ACR
-

Real-Time Architecture Explanation

“Developers commit code into GitHub. Jenkins pipeline builds Docker image using Dockerfile, tags it with build number, pushes image to AWS ECR, and Kubernetes pulls image during deployment into EKS cluster.”

Dockerfile Instructions (Very Important)

1. FROM

Defines base image.

FROM openjdk:17

Why important?

Everything starts from base image.

Interview Explanation

“FROM defines the parent/base image on top of which application image is built.”

2. WORKDIR

Sets working directory inside container.

WORKDIR /app

Equivalent to:

cd /app

Why Use WORKDIR?

- Better readability
 - Organized structure
 - Avoid absolute paths
-

3. RUN

Executes command during image build.

RUN apt-get update

Important Point

RUN executes at build time.

Not container runtime.

Real-Time Example

RUN mvn clean package

Builds application during image creation.

4. CMD

Default command when container starts.

CMD ["java","-jar","app.jar"]

Can be overridden.

CMD vs RUN

RUN	CMD
Build time	Runtime
Executes during image build	Executes during container startup

Interview Answer

“RUN is used during image creation, whereas CMD defines default startup command for the container.”

5. ENTRYPOINT

Defines main executable.

ENTRYPOINT ["java","-jar","app.jar"]

Harder to override.

CMD vs ENTRYPOINT

CMD	ENTRYPOINT
-----	------------

Easily overridden	Fixed main process
Optional args	Main executable

Real-Time Usage

ENTRYPOINT:

ENTRYPOINT ["python"]

CMD:

CMD ["app.py"]

Final execution:

python app.py

6. COPY

Copies files from local system to container.

COPY . .

Real-Time Example

COPY target/app.jar /app/

7. ADD

Similar to COPY but supports:

- URL download
 - Auto extraction of tar files
-

Best Practice

Prefer COPY unless ADD features are needed.

8. EXPOSE

Documents container port.

EXPOSE 8080

Does NOT automatically publish port.

Complete Real-Time Dockerfile

```
FROM openjdk:17
```

```
WORKDIR /app
```

```
COPY target/app.jar .
```

```
EXPOSE 8080
```

```
ENTRYPOINT ["java","-jar","app.jar"]
```

Real-Time Docker Build Flow

Code

↓

Dockerfile

↓

docker build

↓

Docker Image

↓

Push to Registry

↓

docker run / Kubernetes Deployment

1. Container Exited Suddenly

Very common production issue.

What It Means?

Main process inside container stopped.

When PID 1 exits → container stops.

Troubleshooting Steps

Step 1: Check Container Status

`docker ps -a`

Step 2: Check Logs

`docker logs container-id`

Most important command.

Common Reasons

Application Crash

`NullPointerException`

Wrong CMD/ENTRYPOINT

Example:

`CMD ["app.jar"]`

Instead of:

```
CMD ["java","-jar","app.jar"]
```

Missing Dependencies

ModuleNotFoundError

Port Conflict

Application unable to bind port.

Interview Answer

“If container exits suddenly, first I check docker logs to identify whether application crashed or startup command failed. In one issue, container exited immediately because Java runtime was missing in the base image. We corrected the Dockerfile base image and rebuilt successfully.”

2. Application Working Locally But Not in Container

Most asked Docker troubleshooting question.

Why This Happens?

Local machine has:

- Installed dependencies
- Environment variables
- Different OS behavior

Container may not.

Common Reasons

Missing Environment Variables

`docker run -e DB_HOST=mysql`

Wrong Port Binding

Container runs on:

8080

But application exposed on:

3000

Dependency Missing

Package installed locally but absent in image.

File Permission Issue

Linux container permissions issue.

Debugging Commands

`docker exec -it container-id sh`

Enter inside container.

Interview Answer

“If application works locally but not inside container, I usually verify environment variables, dependencies, ports, and runtime configurations. I also enter container using docker exec for debugging and compare runtime differences.”

3. Container Restarting Continuously

What It Means?

Application inside container keeps failing.

Docker/Kubernetes restarts container repeatedly.

Similar Kubernetes Term

CrashLoopBackOff

Common Reasons

- App crash
 - DB connectivity issue
 - Wrong startup command
 - Health check failure
 - Resource exhaustion
-

Troubleshooting

docker logs
docker inspect
docker stats

Real-Time Example

“Container was restarting continuously because DB hostname configured in environment variable was incorrect. Application failed during startup while connecting to database.”

4. High Memory Usage

Very important production issue.

Causes

Memory Leak

Application consuming memory continuously.

Large JVM Heap

Example:

-Xmx4G

Too Many Processes

Multiple unnecessary services running.

Large Docker Image

Heavy base image.

Troubleshooting Commands

`docker stats`

Shows:

- CPU
 - Memory
 - Network
-

Optimization Techniques

Use Lightweight Images

`FROM alpine`

Multi-Stage Build

Reduces image size.

Example

`FROM maven AS build`

`RUN mvn package`

`FROM openjdk`

`COPY --from=build app.jar .`

Interview Answer

“We faced high memory usage in Java containers because heap size was overallocated. We optimized JVM memory settings and implemented lightweight base images, reducing overall container resource consumption.”

5. Container Communication Issue

Very important Docker networking question.

Example Scenario

Frontend container unable to communicate with backend container.

Common Reasons

Containers Not in Same Network

Check:

`docker network ls`

Wrong Service Name

Using wrong hostname.

Port Not Exposed

Backend service not listening.

Debugging

`docker inspect`

`ping`

`curl`

`netstat`

Real-Time Solution

docker network create app-network

Attach containers to same network.

Interview Answer

“In one issue, frontend container couldn’t connect to backend because both containers were deployed in different Docker bridge networks. We attached them to a common custom network and communication started working properly.”

Important Docker Commands (Must Know)

docker ps

Shows running containers.

docker ps

docker ps -a

Shows all containers.

docker images

Lists images.

docker build

Builds image.

```
docker build -t app:v1 .
```

docker run

Runs container.

```
docker run -d -p 8080:8080 app:v1
```

docker logs

Shows container logs.

```
docker logs container-id
```

docker exec

Access running container.

```
docker exec -it container-id sh
```

docker inspect

Detailed metadata.

```
docker inspect container-id
```

docker stats

Live resource monitoring.

```
docker stats
```

docker rm

Removes container.

docker rm container-id

docker rmi

Removes image.

docker stop

Stops container.

docker start

Starts stopped container.

Most Important Real-Time Scenario Answer

If interviewer asks:

“Explain how Docker is used in your project”

Say this:

“In my project, developers push code to GitHub, Jenkins pipeline triggers automatically, application gets built using Maven, Docker image is created using Dockerfile, image is scanned using Trivy, pushed to AWS ECR, and deployed into Kubernetes EKS cluster using Helm charts. Docker helped us maintain consistent runtime environments across development, QA, and production.”